

Subject Name: OBJECT ORIENTED PROGRAMMING DESIGN

Subject Code: CS T33:

2 MARKS AND 11 MARKS

UNIT IV

Files: File Stream classes – Checking for errors – file opening modes – file pointers and manipulators – manipulators with arguments – read and write operations – Binary and ASCII files – Random access operation – Error handling functions – command line arguments – stdstreams.

Generic Programming with Templates: Generic Functions- Need of Template – Normal function template – class template with more parameters – Function template with more parameters, overloading of function templates, class template with overloaded operators – class templates and inheritance.

Exception Handling: Fundamentals of Exception Handling – Catching Class Types – Using Multiple catch statements – Catching All Exception – Rethrowing Exception – Specifying Exception – Exceptions in constructors and destructors – controlling uncaught Exceptions – Exception and operator overloading – Exception and inheritance – Class Template and Exception handling.

PART A – 2 MARK QUESTIONS

1. What are the various traditional error handling methods?
--

The various traditional error handling methods are,

- i. Returning error number.
 - ii. Global flag manipulation.
 - iii. Abnormal termination.
-

2. What is the importance of exceptional handling?

The importance of exceptional handling is,

- i. Divide the error handling.
- ii. To provide unconditional termination and programmer preferred termination
- iii. For separating error reporting and error handling.
- iv. To solve the object destroy problem.

3. What are the three keywords used for exception handling mechanism?

The three keywords used for exception handling mechanism are,

- i. try : for indicating program area where the exception can be thrown.
- ii. throw: for throwing an exception.
- iii. catch: for taking an action for specific exception.

4. What is the use of unexpected function?

The unexpected function is called when a function throws an exception not listed in its exception specification. Internally, it calls the terminate function to terminate the program execution. The function set_unexpected() is used to call our own unexpected function in place of the built-in unexpected function.

5. What are the challenges in the exception handling mechanism?

The challenges in the exception handling mechanism are,

- i. Finding proper catch block.
- ii. Finding proper catch block for polymorphic object.
- iii. Backtracking till the beginning of the try block.

6. Why Templates are used in C++?

The Templates are used to develop reusable software component such as functions, classes, etc. Templates allow the construction of a family of templates functions and classes to perform the same operations on different data types.

7. What are rules for invalid function template declaration?

The rules for invalid function template declaration are,

- i. No-argument template function.
- ii. Template-type argument unused.
- iii. Usage of partial number of template arguments.

8. Write the syntax for function Template.

The syntax for function Template is,
Template <class T, >
Return Type Fun_Name (arguments)
{
..... // body of the template
}

9. What are the error handling function in C++?

The error handling function in C++ is,

- i. eof()
- ii. fail()
- iii. bad()
- iv. good()

10. What are the rules for virtual function?

The rules for virtual function are,

- i. They cannot be static members.
- ii. They are access by using object pointers.
- iii. A virtual function can be a friend of another class.

11. What are Streams?

Stream is a mechanism, which supplies the input data to a program and presents the processed data in the desired form.

12. What are the file stream classes in C++?

The file stream classes in C++ are,

- i. filebuf
- ii. fstreambase
- iii. ifstream
- iv. ofstream
- v. fstream

13. List out the ways of Class Template inheritance.

The ways of Class Template inheritance are,

- i. Derive a class template from a base class, which is a template class
- ii. Derive a class template from the base class, which is a template class, add more template members in the derived class
- iii. Derive a class template from a base class which is not a template class, and add template members to that class
- iv. Derive a class template from a base class which is a template class and restrict the template feature.

14. What are the functions of ios classes?

The functions of ios classes are,

Function

width()

Task Performed

Specifies the required number of fields to be used while displaying the output value.

precision()

Specifies the number of digits to be displayed after the decimal point

fill()

Specifies a character to be used to fill the unused area of a field.

setf()

Sets format flag that control the form of output display

unset()

Clears the specified flag

15. What are the modes used file stream class?

The modes used file stream class is,

- i. in – open for reading
- ii. out – open for writing
- iii. app – all write occur at end of file
- iv. trunc – truncate the file if already exists
- v. nocreate – open fails if file does not exists
- vi. noreplace – open fails if file already exists
- vii. binary – open as a binary file

16. What is get () and put () functions?

get() is a member function of a stream class fstream and is used to read a single character from a file. put() is a member function of a stream class fstream and is used to write a single character to the file.

17. What are two types of file pointers used in C++?

The two types of file pointers used in C++ are,

- i. get pointer – specifies a location from where the current reading operation is initiated
- ii. put pointer - specifies a location from where the current writing operation is initiated

18. Define template.

A template in C++ allows the construction of a family of template functions and classes to perform the same operation on different data types. The template type arguments are called “generic data types”.

19. DEFINE CLASS TEMPLATE.

The templates declared for classes are called class templates. Classes can also be declared to operate on different data types. A class template specifies how individual classes can be constructed similar to normal class specification.

20. What are the types of exceptions? C++ provides what type of exception(MAY 2007)

Exceptions are of two types. They are

- a. Synchronous exceptions
- b. Asynchronous exceptions

C++ has a well-organized object oriented method to control run-time errors that occur in the program. The goal of exception handling is to create a routine that detects and sends an exceptional condition in order to execute suitable action. The routine needs to carry following responsibilities:

- a. Detect the problem
- b. Warn that an error has come
- c. Accept the error message
- d. Perform accurate actions without troubling the user.

21. Define Exception. (MAY 2011)

An exception is an object. It is sent from the part of the program where an error occurs to that part of the program which is going to control the error. Exception provides an explicit pathway that contains errors to the code that controls the errors.

22. What is a synchronous exception and asynchronous exception?

Synchronous exception: The technique that is not suitable to handle the current class of data, within the program are known as synchronous exception.

Asynchronous Exception: The exceptions caused by events or faults unrelated to the program and beyond the control of program are called asynchronous exceptions.

23. Write the syntax of try construct

The try keyword defines a boundary within which an exception can occur. A block of code in which an exception can occur must be prefixed by the keyword try. Following the try keyword is a block of code enclosed by braces. This indicates that the program is prepared to test for the existence of exceptions

```
try
{
    //code raising exception or referring to a function raising exception
}
catch(type_id1)
{
    //actions for handling an exception
}
...
...
catch(type_idn)
{
    //actions for handling an exception
}
```

24. Define catch construct.

The exception handler is indicated by the keyword catch. It must be used immediately after the statements marked by the keyword try. Each catch handler will evaluate an exception that matches to the specified type in the argument list.

Syntax:

```
Catch (T) // named object or nameless object
{
    Actions for handling an exception
}
```

25. Define exception specification

It is possible to specify what kind of exception can be thrown by functions, using a specific syntax. We can append the function definition header with throw keyword and possible type of expressions to be thrown in the parenthesis. It is known as exception specification.

26. What are the two types of errors or bugs?

- a) Logical errors-errors occur due to poor understanding of the problem and solution procedure
- b) Syntactic errors-errors that arise due to poor understanding of the language itself.

27. Write the functions to handle uncaught exceptions
--

- 1) terminate()
- 2) set_terminate()
- 3) unexpected()
- 4) set_unexpected()

PART-B(11 MARKS)

1. Explain in detail about checking for errors in files
--

If any fault occurs during file operation, fault cannot be detected. Various errors can be made by the user while performing file operation. Such errors must be reported in the program to avoid further program failure. When the user attempts to read file that does not exist or opens a read-only file for writing purpose, in such situations operation fails. Such errors must be reported and proper actions have to be taken before further operations.

The !(logical negation operators) overloaded operator is useful for detecting the errors. It is a unary operator and in short it is called as not operator. The (!) not operator can be used with object of stream classes. This operator returns non-zero value if stream error is found during operation.

Write a program to check whether the file is successfully opened or not

```
#include<fstream.h>

#include<constream.h>

void main()
{
clrscr();

ifstream in("text");

if(!in)

cerr<<"\n File is not opened"<<in;

else

cerr<<"\n file is opened"<<in;

}
```

OUTPUT

File is opened 0x8f15ffd2

Explanation: in the above program, an existing file “text” is opened for reading. The file is opened successfully. Hence, no error is generated. The if() statement checks the contents in object using the following statement if(!in). The value of object is 0x8f15ffd2. In case the operation fails the value of object in would be 0x8f160000.

Write a program to detect error in file operation using ! operator.

```
#include<fstream.h>

#include<constream.h>

#include<iomanip.h>
```



```
main()
{
clrscr();
char c, f_name[10];
cout<<"\n enter file name:";
cin>>f_name;
ifstream in(f_name);
if(!in)
{
cerr<<"Error in opening file "<< f_name <<endl;
return 1;
}
in>>resetiosflags(ios::skipws);
while(in)
{
in>>c;
cout<<c;
}
return 0;
}
```

OUTPUT

Enter file name : TEXT

One two three four

1 2 3 4

** the end **

Explanation:

In the above program, using constructor of a class ifstream a file is opened. The file that is to be opened is entered by the user during program execution. If the file does not exist, the ifstream object(in) contains 0. The if() statement checks the value of object in and if operation fails a message will be displayed and program is terminated. In case the file exists then using while() loop, contents of file is read one character at a time and displayed on the screen.

2. Explain in detail about file opening modes
--

The opening of file also involves several modes depending upon operation to be carried out with the file. The open() function has two arguments as given below:

Syntax of open() function

```
object.open("filename",mode);
```

File Modes

Mode parameter	Operation
ios::app	Adds data at the end of file
ios::ate	After opening character pointer goes to the end of file
ios::binary	Binary file
ios::in	Opens file for reading operation
ios::nocreate	Opens unsuccessful if the file does not exist
ios::noreplace	Opens file if its already present
ios::out	Opens file for writing operation
ios::trunc	Erases the file contents if the file is present.

- The mode ios::out and ios::trunc are the same. When ios::out is used, the contents of specified file(if present) will be deleted(truncated). The file is treated as new file.

- When file is opened using `ios::app` and `ios::ate` modes, the character pointer is set to end of the file. The `ios::app` lets the user to add data at the end of file whereas `ios::ate` allows user to add or update data anywhere in the file. If the given file does not exist, new file is created. The mode `ios::app` is applicable to the output file only.
- The `ifstream` creates input stream and `ofstream` creates output stream. Hence, it is not compulsory to give mode parameters.
- While creating an object of `fstream` class, the programmer should provide the mode parameter. The `fstream` class does not have default mode.
- The file can be opened with one or more mode parameters. When more than one parameters are necessary, bitwise or operator separates them. The following statement opens a file for appending. It does not create a new file if the specified file is not present.

File opening with multiple attributes

```
out.open("file1", ios::app | ios::nocreate)
```

//PROGRAM TO OPEN A FILE FOR WRITING AND STORE FLOAT NUMBERS IN IT

```
#include<fstraem.h>
#include<iomanip.h>
void main()
{
float a=76.3, b=34.9,c=12.3;
ofstream out("float.txt",ios::trunc);
out<<setw(10)<<a<<endl;
out<<setw(10)<<b<<endl;
out<<setw(10)<<c<<endl;
}
```

truncate

756.299988

3455.899902

1222.300049

Program to open a file in binary mode. Write and read the data.

```
#include<fstream.h>

#include<conio.h>

int main()
{
    clrscr();

    ofstream out;

    char data[32];

    out.open("text", ios::out | ios::binary);

    cout<<"\n enter text"<<endl;

    cin.getline(data, 32);

    out<<data;

    out.close();

    ifstream in;

    in.open("text", ios::in | ios::binary);

    cout<<endl<<"contents of the file ";

    char ch;

    while (in.eof()!=0)
    {

        ch = in.get();
```

```

cout<<ch;

}

return 0;

}

```

OUTPUT

Programming in ANSI and TURBO-C

Contents of the file

Programming in ANSI and TURBO-C

3. Explain in detail about file pointers and manipulators

All file objects holds two file pointers associated with the file. These two file pointers provide two integer values. These integer values indicate exact position of file pointers in number of bytes in the file. The read or write operations are carried out at the location pointed by these file pointers. One of them is called as get pointer(input pointer) and the second one is called as put pointer (output pointer). During reading and writing operations with files, these file pointers are shifted from one location to another location in the file. The (input) get pointer helps in reading the file from the given location and the output pointer helps for writing data in the file at specified location. When read and write operations are carried out, respective pointer is moved.

While file is opened for reading or writing operation, the respective file pointer, input or output is by default set at the beginning of the file.

Read mode

When a file is opened in red mode, the get pointer is set at the beginning of the file. Hence, it is possible to read the file from the first character of the file.



Write mode

When a file is opened in write mode, the put pointer is set at the beginning of the file. It allows write operation from beginning of the file. In case the specified file already exists, its contents will be deleted.

Append mode

This mode allows to add data at the end of file. When file is opened in append mode, the output pointer is set at the end of file. Hence, it is possible to write data at the end of file. In case the specified file already exists, new file is created, and the output is set at the beginning of the file. When a pre-existing file is successfully opened in append mode, its contents remain safe and new data is appended at the end of file.

File pointer handling functions

Function	Uses	Remark
seekg()	Shifts input(get) pointer to a given location	Member of ifstream class
seekp()	Shifts output(put) pointer to a given location	Member of ofstream class
tellg()	Provides the present position of the input pointer	Member of ifstream class
tellp()	Provides the present position of the outputpointer	Member of ofstream class

Program to append file

```
#include<fstream.h>
```

```
#include<conio.h>
```

```
int main()
```

```
{
```

```
clrscr();
```

```
ofstream out;
```

```
char data[25];
```

```
out.open("text", ios::out);

cout<<"enter text"<<endl;

cin.getline(data,25);

out<<data;

out.close();

out.open("text", ios::app);

cout<<"\n Again enter text "<<endl;

cin.getline(data,25);

out<<data;

out.close();

ifstream in;

in.open("text", ios::in);

cout<<endl<<"contents of the file\n";

while(in.eof()!=0)

{

in>>data;

cout<<data;

}

return 0;

}
```

OUTPUT

Enter text

C-PLUS-

Again enter text

PLUS

Contents of the file

C-PLUS-PLUS

Explanation:

In the above program, the file text is opened for writing output. The text read through the keyboard is written in the file. The close() function closes the file. Once more, the same file is opened in append mode and data entered through the keyboard is appended at the end of file. The append mode allows the programmer to write data at the end of file. The close() function closes the file. The same file is opened using the object of ifstream class for reading purpose. The while loop is executed until the end of file is detected. The statements within while loop reads text from file and displays it on the screen.

Program to read contents of file. Display the position of the get pointer.

```
#include<fstream.h>

#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[32];
    out.open("text", ios::out);
    cout<<"\n enter text"<<endl;
    cin.getline(data,32);
    out<<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"contents of file\n";
    int r;
    while(in.eof()!=0)
    {
        in>>data;
        cout<<data;
```



```
r=in.tellg();  
cout<<"("<<r<<")";  
  
}  
  
return 0;  
  
}
```

OUTPUT

Enter text

Programming in ANSI and TURBO-C

Contents of file

Programming (11)In (14)ANSI (19) and (23)TURBO-C(31)

Explanation:

The above program is same as the previous one. In addition here, the function tellg() is used. This function returns current file pointer position in number of bytes from beginning of the file. The number shown in brackets in output specifies position of file pointer from the beginning of file.

4. Explain in detail about file manipulators with arguments
--

The seekp() and seekg() functions can be used with two arguments. Their format with two arguments follows below:

```
seekg(offset, pre_position);
```

```
seekp(offset, pre_position);
```

The first argument offset specifies the number of bytes the file pointer to be shifted from the argument pre_position of the pointer. The offset must be a positive or negative number. The negative number moves the pointer in forward direction whereas positive number moves the pointer in backward direction. The pre_position argument may have one of the following values.

ios::beg Beginning of the file

ios::cur Current position of the file pointer

ios::end End of the file

File Pointer with its arguments

Seek option	Working
in.seekg(0,ios::beg)	Go to the beginning of file
in.seekg(0,ios::cur)	Rest at the current position
in.seekg(0,ios::end)	Go to the end of file
in.seekg(n,ios::beg)	Shifts the pointer to n+1 byte in the file
in.seekg(n,ios::cur)	Go front by n byte from current position
in.seekg(-n,ios::cur)	Go back by n bytes from the present position
in.seekg(-n,ios::end)	Go back by n bytes from the end of file

Program to write text in the file. Read the text from the file from end of file. Display the contents of file in reverse order.

```
#include<fstream.h>

#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[25];
    out.open("text",ios::out);
    cout<<"\n Enter text "<<endl;
    cin.getline(data,25);
    out<<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"Reverse contents of file";
```

```
in.seekg(0,ios::end);

int m=in.tellg();

char ch;

for(int i=1;i<=m;i++)

{

in.seekg(-I,ios::end);

in>>ch;

cout<<ch;

}

return 0;

}
```

OUTPUT

Enter text

Visual_C_+_+

Reverse contents of the file

+_+_lausiv

Explanation:

In the program, text file is opened in output mode and string is written to the file. Again the same file is opened for reading purpose. The statement `in.seekg(0,ios::end);` moves the get pointer at the end of file. The `tellg()` function returns the current position of the file pointer in the file. Hence, file pointer is set to the end of file. The `tellg()` returns the number of last byte. The for loop executes from 1 to m. the statement `in.seekg(-i, ios::end)` reads ith byte from the end of file. The statement `in>>ch` reads the character from file indicated by the file pointer. The `cout` statement displays the read character on the screen. Thus, the contents of the file displayed in reverse order.

5. Explain in detail about sequential read and write operations
--

C++ allows file manipulation command to access file sequentially or randomly. The data of sequential file must be accessed sequentially i.e. one character at a time. In order to access nth number of bytes, all previous characters are read and ignored. There are number of functions to perform read and write operations with the files. The put() and get() functions are used to read or write a single character whereas write() and read() are used to read or write block of binary data.

The put() and get() functions

The function get() is a member function of the class fstream. This function reads a single character from the file pointed by the get pointer. i.e the character at current get pointer position is caught by get() function.

The put() function writes a character to the specified file by the stream object. It is also a member of fstream class. The put() function places a character in the file indicated by put pointer.

```
#include<iostream.h>

#include<conio.h>

#include<string.h>

int main()

{

clrscr();

char text[50];

cout<<"enter a text";

cin.getline(text,50);

int l=0;

fstream io;

io.open("data", ios::in | ios::out);

while(l[text]!='\0')

io.put(text[l++]);

io.seekg(0);

char c;

cout<<"\n entered text:";

while(io)

{
```

```
io.get(c);

cout<<c;

}

return 0;

}
```

OUTPUT

Enter a text: PROGRAMMING WITH ANSI AND TURBO-C

Entered text: PROGRAMMING WITH ANSI AND TURBO-C

Explanation:

In the above program, the file data is opened in read and write mode at once. The `getline()` function reads the string through the keyboard and stores in the array `text[50]`. The statement `io.put(text[l++])` in the first while loop reads one character from array and writes it to the file indicated by the stream object `io`. The first while loop terminates when the null character is found in the text. The statement `io.seekg(0)` sets the file pointer at the beginning of the file. In the second while loop, the statement `io.get(c)` reads one character at a time from the file and `cout()` statement displays the same character on the screen. The while loop terminates when end of file is detected.

BINARY AND ASCII FILES

The insertion and extraction operators, known as stream operator, handles formatted data. The programmer needs to format data in order to represent it in a suitable fashion. The description of formatted and unformatted data is given in chapter “INPUT and OUTPUT IN C++”. ASCII codes are used by I/O devices to share or pass data to the computer system, but central processing unit(CPU) manipulates the data using binary numbers, i.e., 0 and 1. For this reason, it is essential to convert the data while accepting data from input devices and displaying the data on output devices. Consider the following statements:

```
cout<<k; //Display value of k on screen

cout<<k; //reads value for k from keyboard
```

here, `k` is an integer variable. The operator `<<` converts value of integer variable `k` into stream of ASCII characters. In the same fashion, the `<<` operator converts the ASCII characters entered by the user to binary. The data is entered through the keyboard, a standard input device. For

example, if 21 is entered, the stream operator >> gets ASCII codes of the individual digits of the entered number 21 i.e., 50 and 49. ASCII code of 2 and 1 are 50 and 49 respectively.

//program to read data from file using ASCII format

```
#include<iostream.h>

#include<constream.h>

int main()

{

clrscr();

char c;

ifstream in("dat");

if (!in)

{

cerr<<" Error in opening file.";

return 1;

}

while(in.eof()==0)

{

cout<<(char)in.get();

}

return 0;

}
```

Output

PROGRAMMING WITH ANSI AND TURBO-C

Explanation

In the above program, the dat file is opened in read mode. The file already exists. Using get() member function of ifstream class, the contents of file read and displayed. Consider the following statement:

```
Cout<<(char)in.get();
```

The get() function reads data from file in ASCII format. Hence, it is necessary to convert the ASCII number to equivalent character. The typecasting format(char) convert ASCII number to equivalent character.

ERROR HANDLING FUNCTIONS

Status bits

Eofbit	End of file encountered	0x01
Failbit	Operation unsuccessful	0x02
Badbit	Illegal operation due to wrong size of buffer	0x04
hardfail	Critical error	0x08

Error trapping functions

Functions	Working and return value
fail()	Returns non-zero value if an operation is unsuccessful. This is carried out by reading the bits ios::failbit, ios::badbit, and ios::hardfail of ios::state
eof()	Returns non-zero value when end of file is detected otherwise returns zero. The ios::eofbit is checked.
bad()	Returns non-zero value when error is found in operation. The ios::badbit is checked
good()	Returns non-zero value if no error occurred during file operation
Rdstate()	Returns the stream state. It returns value of various bits of ios::state

Program to detect whether the file is opened successfully or not.

```
#include<fstream.h>
```

```
#include<constream.h>
```

```
#include<string.h>
```

```
ifstream in;
```

```
void main()
{
clrscr();
void show();
in.open("dat");
char c;

if(in!=0)
show();
else
cout<<"\n File not found";
}
void show()
{
char c;
cout<<"\n Contents of file:";

while(in)
{
in.get( c );
cout<<c;
}
}
```

Output

File not found

Explanation:

In the above program object in of ifstream class is declared globally. It can be accessed by any normal function. In function main() using the object in a file is opened. If the open() function fails to open file, it returns zero otherwise non-zero value. The if statement checks the value of object in and if it is non-zero show() function is invoked, otherwise file not found message is displayed. The show() function reads file and displays the contents on the screen. In the above program, the output is “file not found” . if the specified file exists, the contents of file will be displayed.

Write a program to display status of various error trapping functions.

```
#include<fstream.h>

#include<conio.h>

void main()

{

clrscr();

ifstream in;

in.open(“text.txt”,ios::nocreate);

if(!in)

cout<<“File not found”;

else

cout<<“\n File=”<<in;

cout<<“\n Enter state=”<<in.rdstate();

cout<<“\n good()=”<<in.good();

cout<<“\n eof()=”<<in.eof();

cout<<“\n fail()=”<<in.fail();

cout<<“\n bad()=”<<in.bad();

}
```

Output

File not found

Error state =4

Good() =0

Eof() =0

Fail() =4

Bad() =4

Explanatins:

In the above program, an attempt is made to open a non-existent file. The if statement checks the value of object in. the specified file does not exist hence it displays the message “file not found” the program also displays the values of various bits using the functions good(), eof(), bad(), fail() and rdstate() error trapping function.

COMMAND LINE ARGUMENTS

An executable program that performs a specific task for operating system is called as command. The commands are issued from the command prompt of operating system. Some arguments are to be associated with the commands. Hence these arguments are called as command line arguments. These associated arguments are passed to program.

The main() function can receive two arguments and they are argc(argument counter) and argv(argument vector). The first argument contains the number of arguments and second argument is an array of char pointers. The *argv points to the command line arguments. The size of array is equal to value counted by the argc. The information contained in the command line is passed on to the program through these arguments when the main() is called up by the system.

Argument argc: an argument argc counts total number of arguments passed from command prompt. It returns a value that is equal to total number of arguments passed through the main().

Argument argv: it is a pointer to an array of character strings that contain names of arguments. Each word is an argument.

Syntax: main(int argc, char *argv[])

Example : ren file1 file2.

Here file1 and file2 are arguments and copy is a command. The first argument is always an executable program followed by associated arguments. If argument is not the first program name itself becomes an argument but the program will not run properly and will flag an error. The contents of argv[] would be as per following:

argv[0] → ren

argv[1] → file1

argv[2] → file2

Program to simulate rename command using command line arguments

```
#include<stdio.h>
```

```
#include<fstream.h>
```

```
#include<conio.h>
```

```
#include<process.h>
```

```
main(int argc, char *argv[])
```

```
{
```

```
    fstream out;
```

```
    ifstream in;
```

```
    if(argc<3)
```

```
    {
```

```
        cout<<"Insufficient Arguments";
```

```
        exit(1);
```

```
    }
```

```
    in.open(argv[1],ios::in|ios::nocreate);
```

```
    if(in.fail())
```

```
    {
```

```
        cout<<"\n File not found";
```

```
        exit(0);
```

```
    }
```

```
    in.close();
```

```
    out.open(argv[2],ios::in|ios::nocreate);
```

```
    if(out.fail())
```

```
    {
```

```
        rename(argv[1],argv[2]);
```

```
}  
  
else  
  
cout<<"\n Duplicate file name or file is in use";  
  
return 0;  
  
}
```

Explanation:

In the above program, the main() receives two file names. The existence of file can be checked by opening it in read mode. If the file does not exist the program terminates. On the other hand if the second file exists the rename operation cannot be done. The rename operation is carried out only when first file exists and the second file does not exist. Make exe file of this program and use it on the command prompt.

STRSTREAMS

(1) Ostrstream

The strstream class is derived from istream and ostream classes. The strstream class works with memory. Using objects of ostream class different type of data values can be stored in an array.

Program to demonstrate use of ostream objects.

```
#include<strstream.h>  
  
#include<iomanip.h>  
  
#include<conio.h>  
  
main()  
{  
  
clrscr();  
  
char h='C';  
  
int j=451;  
  
float PI=3.141;  
  
char txt[]="applications";  
  
char buff[70];  
  
ostream o (buff,0);
```

```
o<<endl<<setw(9)<<"h="<<h<<endl<<setw(9)<<"j="<<oct<<j<<endl
<<setw(10)<<"PI="<<setiosflags(ios::fixed)<<PI<<endl<<setw(11)
<<"txt="<<txt<<ends;

cout<<o.rdbuf();

return 0;

}
```

Output

h=C

j=703

PI=3.141

txt=applications

Explanation:

The stringstream deals with memory. If we want to pick characters from a stringstream or we want to add characters into stringstream, this can be done by creating istream and ostream objects. When object o is created the constructor of ostream is executed. Once an object of ostream is created we can assign any formatted text to array associated with it. The statement `cout<<o.rdbuf()` displays the formatted information on the screen.

(2) Istream

It is one of the base classes of the stringstream class. Using objects of istream class, data can be extracted from an array.

```
#include<sstream.h>

#include<conio.h>

main()
{

clrscr();

char *book;
```

```
int pages;

float price;

char *text="550 175.76 C++";

istream o(text);

o>>pages>>price>>book;

cout<<endl<<pages<<endl<<price<<endl<<book;

cout<<o.rdbuf();

return 0;

}
```

Output

550

175.75

C++

Explanation:

The istream is opposite of the ostream. It picks different types of data from an array. In the above program, character pointer text contains data of integer, float and character type. Using the object of istream class we can separate the contents and store them in appropriate variable. The *book pointer variable displays string. The remaining contents are displayed by the function rdbuf().

Class template with more parameters

Classes can be declared to handle different data types. Such classes are known as class templates. These classes are generic type and member functions of these classes can operate on different data types. The class template may contain one or more parameters of generic data type. The arguments are separated by comma with template declaration. The declaration is as follows:

Template with multiple parameters

```
template <class T1, class T2>
```

```
class name_of_class
```

```
{
```

```
// class declarations and definitions
```

```
}
```

Write a program to define a constructor with multiple template variables.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
template <class T1, class T2>
```

```
class data
```

```
{
```

```
public:
```

```
data(T1 a, T2 b)
```

```
{
```

```
cout<<"\n a = "<<a<<"\n b = "<<b;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
clrscr();
```

```
data <int,float> h(2,2.5);
```

```
data<int, char> i(15,'c');
```

```
data<float,int> j(3.12,50);
```

```
return 0;
```

```
}
```

OUTPUT:

a = 2 b = 2.5

```
a = 15 b = c
```

```
a = 3.12 b = 50
```

Explanation:

In the above program, the data constructor has two arguments of generic type. While creating objects, type of arguments is mentioned in the angle bracket. When arguments are more than one they are separated by comma. Consider the statement `data<int, float> h(2,2.5)`. It tells the compiler that the first argument is of integer type and second argument is of float type. When objects are created, constructors are called and values are received by the template arguments.

Function Templates with more arguments

Function template is similar to class template. The format is given below:

```
template <class T>
return_data_type function_name(parameter of template type)
{
Statement1;
Statement 2;
Statement 3;
}
```

Write a program to display the elements of integer and float array using template variables with member function.

```
#include<iostream.h>
#include<conio.h>
template <class T1, class T2>
class data
{
```



```
public:
void show(T1 a, T2 b)
{
cout<<"\n a = "<<a<<"\n b = "<<b;
}
};

int main()
{
clrscr();

int i[] = {3,5,2,6,8,9,3};
float f[] = {3.1,5.8,2.5,6.8,1.2,9.2,4.7};

data <int,float> h;

for(int m=0;m<7;m++)
h.show(i[m], f[m]);

return 0;
}
```

OUTPUT

a = 3 b = 3.1

a = 5 b = 5.8

a = 2 b = 2.5

a = 6 b = 6.8

a = 8 b = 1.2

a = 9 b = 9.2

a = 3 b = 4.7

Explanation:

In the above program, array elements of integer and float are passed to function show(). The function show() has two arguments of template type i.e, a and b. the show() function receives integer and float values and displays them.

Overloading of Template functions

a template function also supports overloading mechanism. It can be overloaded by normal function or template function. While invoking these functions an error occurs if no accurate match is met. No implicit conversion is carried out in parameters of template functions. The compiler follows the following rules for choosing appropriate function when program contains overloaded functions.

- a. Searches for accurate match of functions; if found it is invoked.
- b. Searches for a template function through which a function that can be invoked with accurate match can be generated; if found it is invoked.
- c. Attempts normal overloading declaration for the function.
- d. In case no match is found, an error will be reopened.

Write a program to overload a template function

```
#include<iostream.h>

#include<conio.h>

template <class A>
void show(A c)
{
    cout<<"\n Template variable c="<<c;
}

void show(int f)
{
    cout<<"\n Integer variable f="<<f;
}

};

int main()
```

```
{  
clrscr();  
show('C');  
show(50);  
show(50.25);  
return 0;  
}
```

Output

Template variable c=C

Integer variable f=50

Template variable c=50.25

Explanation

In the above, the function show() is overloaded. One version contains template arguments and other version contains integer variables. In main(), the show() function is invoked three times with char, int and float and values are passed. The first call executes the template version of function show(), the second call executes integer version of function show() and third call again invokes the template version of function show().

CLASS TEMPLATE WITH OVERLOADED OPERATORS

The template class permits to declare overloaded operators and member function display() is invoked. The fsyntax for creating overloaded operator function is similar to class template members and functions. The following program explains the overloaded operator with template class.

Write a program to overload + operator for performing addition of two template based class objects.

```
#include<iostream.h>  
  
#include<conio.h.  
  
template<class T>
```

```
class num
(
private:
T number;
public:
num()
{
Number=0;
}
void input()
{
cout<<"\n Enter a number:";
cin>>number;
}
Num operator +(num);
Void show()
{
Cout<<number;
}
};

Template <class T>
Num <T> num <T> : : operator + (num <T> c)
{
Num <T> tmp;
Tmp.number=number+c.number;
return(tmp);
```

```
}
```

```
void main()
{
    clrscr();
    num <int> n1,n2,n3;
    n1.input();
    n2.input();
    n3=n1+n2;
    cout<<"\n\t n3=";
    n3.show();
}
```

Output

Enter a number: 8

Enter a number: 4

n3=12

Explanation

In the above program, class num is declared with template variable number. The input() function is used to read number through the keyboard. The operator +() function performs addition of elements of two template objects.

The function main(), n1,n2 and n3 are three objects of num class. The statement n3=n1+n2 invokes the overloaded operator and addition of two objects is performed. The show() function displays the contents of the object.

CLASS TEMPLATES AND INHERITANCE

The templates class can also act as base class. When inheritance is applied is applied with template class it helps to compose hierarchical data structure known as container class.

- (1) Derive a template class and add new member to it. The base class must be of template type.
- (2) Derive a class from non-template class. Add new template type number too derived class.
- (3) It is also possible to derive classes from template base and omit the template feature of the derived classes.

As said earlier, the template characteristics of the base class can be omitted by declaring the type while deriving the class. All the template-based variables are substituted with basic data types. The following declaration illustrates derivation of a class using template featured base class.

```
Template <class TL,...>
class XYZ
{
//Template type data members and member functions
}
```

```
Template <class TL,...>
class ABC:public XYZ <TL,...>
{
//template type data members and member functions
}
```

Write a program to derive a class using template base class.

```
#include<iostream.h>
#include,constream.h>
```

```
Template<class T>
Class one
{
Protected:
T x,y;
Void display(
{
Cout<<x;
}
};
Template<class S.
```

```
Class two: public one<S>
{
    S z;
    Public:
    Two (S a,S b,S c)
    {
        X=a;y=b;
        Z=c;
    }

    Void show()
    {
        Cout<<"\n x="<<x <<"y="<<y<<"z="<<z;
    }
};

Void main()
{
    Clrscr();
    Two <int> i(2,3,4);
    i.show();
    two<float> f(1.1,2.2,3.3);
    f.show();
}
```

Output

x=2 y=3 z=3

x=1.1 y=2.2 z=3.3

Explanation

in the above program the class one is a base class and class two is a template-derived class.
Consider the statement given below:

template <class S>

class two: public one <S>

using the above statement derivation is carried out. The base class name is followed by template class name S. the function show() is a member of derived class two(). In function main(), i and f are objects of class two. The object i is used to pass integer elements and f is used to pass numbers.

1. Describe exception handling.

Exception handling allows you to manage run-time errors in an orderly fashion. Using exception handling, the program can automatically invoke an error-handling routine when an error occurs. The principal advantage of exception handling is that it automates much of the error-handling code that previously had to be coded "by hand" in any large program.

Exception Handling Fundamentals

- C++ exception handling is built upon three keywords: **try**, **catch**, and **throw**.
- Code that you want to monitor for exceptions must have been executed from within a **try** block. (Functions called from within a **try** block may also throw an exception.)
- Exceptions that can be thrown by the monitored code are caught by a **catch** statement, which immediately follows the **try** statement in which the exception was thrown.
- The general form of **try** and **catch** are shown here

```

try {

    // try block

}

catch (type1 arg) {

    // catch block

}

catch (type2 arg) {

    // catch block

}

catch (type3 arg) {

    // catch block

}

..

.

catch (typeN arg) {

```



```
// catch block
```

```
}
```

- When an exception is thrown, it is caught by its corresponding **catch** statement, which processes the exception.
- There can be more than one **catch** statement associated with a **try**.
- Which **catch** statement is used is determined by the type of the exception.
- That is, if the data type specified by a **catch** matches that of the exception, then that **catch** statement is executed (and all others are bypassed).
- When an exception is caught, **arg** will receive its value.
- Any type of data may be caught, including classes that you create.
- If no exception is thrown (that is, no error occurs within the **try** block), then no **catch** statement is executed.

Example: Write a program to throw exception when j=1 otherwise perform the subtraction of x and y.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    int x,y;
    cout<<"\n Enter values of x and y";
    cin>>x>>y;
    int j;
    j=x>y? 0 : 1;
    try
    {
        If(j==0)
        {
            cout<<"subtraction (x-y)="<<x-y<<"\n";
            else
            {
                throw(j);
            }
            catch(int k)
            {
                cout<<"Exception caught: j="<<j<<"\n";
            }
            return 0;
        }
    }
    Output 1:
    Enter values of x and y 7 8
```

Exception caught: j=1

Enter values of x and y 8 4

Output 2:

subtraction (x-y)= 4

Explanation: in the above program, values of x and y are entered. The conditional operator tests the two numbers. If x is greater than y then zero is assigned to j, otherwise one. In the try block the if () condition checks the value of j, the subtraction is carried out when j is zero otherwise the else block throws the exception. The catch statement catches the exception thrown.

2. Explain in detail Catching Class Types

An exception can be of any type, including class types that you create.

Actually, in real-world programs, most exceptions will be class types rather than built-in types. Perhaps the most common reason to define a class type for an exception is to create an object that describes the error that occurred. This information can be used by the exception handler to help it process the error. The following example demonstrates this.

// Catching class type exceptions.

```
#include <iostream>                                     *str_what = 0;

#include <cstring>                                       what = 0;

using namespace std;                                   }

class MyException                                     MyException(char *s, int e)
{
public:                                                {
    strcpy(str_what, s);
    char str_what[80];                                what = e;
    int what;                                         }

    MyException()                                     };
{
```

```

int main()                                { // catch an error

{                                           cout << e.str_what << ": ";

int i;                                     cout << e.what << "\n";

try {                                     }

cout << "Enter a positive number: ";      return 0;

cin >> i;                                }

if(i<0)                                   Output:

throw MyException("Not Positive", i);      Enter a positive number: -4

}                                           Not Positive: -4

catch (MyException e)

```

The program prompts the user for a positive number. If a negative number is entered, an object of the class **MyException** is created that describes the error. Thus, **MyException** encapsulates information about the error. This information is then used by the exception handler. In general, to create exception classes that will encapsulate information about an error to enable the exception handler to respond effectively.

3. Using Multiple catch Statements.

There can be more than one **catch** statement associated with a **try**. In fact, it is common to do so. However, each **catch** must catch a different type of exception. For example, this program catches both integers and strings.

// Different types of exceptions can be caught.

```

#include <iostream>                        try{

using namespace std;                     if(test) throw test;

void Xhandler(int test)                  else throw "Value is zero";

{                                         }

```

catch(int i)	Xhandler(2);
{	Xhandler(0);
cout << "Caught Exception #: " << i << '\n';	Xhandler(3);
}	cout << "End";
catch(const char *str)	return 0;
{	}
cout << "Caught a string: ";	
cout << str << '\n';	Output:
}	Start
}	Caught Exception #: 1
int main()	Caught Exception #: 2
{	Caught a string: Value is zero
cout << "Start\n";	Caught Exception #: 3
Xhandler(1);	End

As you can see, each **catch** statement responds only to its own type. In general, **catch** expressions are checked in the order in which they occur in a program. Only a matching statement is executed. All other **catch** blocks are ignored.

Catching All Exceptions

In some circumstances that we want an exception handler to catch all exceptions instead of just a certain type. This is easy to accomplish. Simply use this form of **catch**.

```
catch(...)
{
    // process all exceptions
}
```

```
}
```

Here, the ellipsis matches any type of data. The following program illustrates **catch(...)**.

// This example catches all exceptions.

```
#include <iostream>

using namespace std;

void Xhandler(int test)
{
    Try
    {
        if(test==0)
            throw test; // throw int

        if(test==1)
            throw 'a'; // throw char

        if(test==2)
            throw 123.23; // throw double

        }

        catch(...)
        {
            // catch all exceptions

            cout << "Caught One!\n";

        } }

int main()
{
    cout << "Start\n";

    Xhandler(0);

    Xhandler(1);

    Xhandler(2);

    cout << "End";

    return 0;
}
```

OUTPUT

```
Start
Caught One!
Caught One!
Caught One!
End
```

As you can see, all three **throws** were caught using the one **catch** statement. One very good use for **catch(...)** is as the last **catch** of a cluster of catches. In this capacity it provides a useful default or "catch all" statement. For example, this slightly different version of the preceding program explicitly catches integer exceptions but relies upon **catch(...)** to catch all others.

// This example uses catch(...) as a default.

```
#include <iostream>

using namespace std;

void Xhandler(int test)

{

try{

if(test==0)

throw test; // throw int

if(test==1)

throw 'a'; // throw char

if(test==2)

throw 123.23; // throw double

}

catch(int i)

{ // catch an int exception

cout << "Caught an integer\n";

}

catch(...)

{ // catch all other exceptions

cout << "Caught One!\n";

}
```

```
}  
  
int main()  
  
{  
  
    cout << "Start\n";  
  
    Xhandler(0);  
  
    Xhandler(1);  
  
    Xhandler(2);  
  
    cout << "End";  
  
    return 0;  
  
}
```

OUTPUT

Start

Caught an integer

Caught One!

Caught One!

End

As this example suggests, using **catch(...)** as a default is a good way to catch all exceptions that you don't want to handle explicitly. Also, by catching all exceptions to prevent an unhandled exception from causing an abnormal program termination.

Rethrowing an Exception

If you wish to rethrow an expression from within an exception handler, you may do so by calling **throw**, by itself, with no exception. This causes the current exception to be passed on to an outer **try/catch** sequence. The most likely reason for doing so is to allow multiple handlers access to the exception. For example, perhaps one exception handler manages one aspect of an exception and a second handler copes with another. An exception can only be rethrown from within a **catch** block (or from any function called from within that block). When you rethrow an exception, it will not be recaught by the same **catch** statement. It will propagate outward to the next **catch** statement. The

following program illustrates rethrowing an exception, in this case a **char *** exception.

// Example of "rethrowing" an exception.

```
#include <iostream>

using namespace std;

void Xhandler()
{
    try {

        throw "hello"; // throw a char *

    }

    catch(const char *) { // catch a char *

        cout << "Caught char * inside Xhandler\n";

        throw ; // rethrow char * out of function

    }

}
```

```
int main()

{

cout << "Start\n";

try{

Xhandler();

}

catch(const char *) {

cout << "Caught char * inside main\n";

}

cout << "End";

return 0;

}
```

OUTPUT:

Start

Caught char * inside Xhandler

Caught char * inside main

End

SPECIFYING EXCEPTION

The specified exceptions are used when we want to bind the function to throw only specified exception. Using a throw list condition can do this. The format for such exception is given below.

Syntax:**data_type function_name(parameter list) throw(data type list)****{****statement 1;****statement2;****}**

The data type list indicates the type of exception to be thrown. If we want to denu a function from throwing exception, we can do this by declaring data type list void as per the following statement.

throw(); //void or vaccant list

Write a program to restrict a function to throw only specified type of exceptions.

```
#include<iostream>
```

```
using namespace std;
```

```
void check(int k) throw (int,double)
```

```
{
```

```
if(k==1) throw 'k';
```

```
else
```

```
if(k==2) throwk;
```

```
else
```

```
If(k==2) throw 1.0;
```

```
cout<<"\n End of function check()";
```

```
}
```

```
void main()
```

```
{
```

```
    cry
```

```
{
```

```
    cout<<"k==1\n";
```

```
    check(1);
```

```
    cout<<"k==2\n";
```

```
    chect(2);
```

```
    cout<<"k==2\n";
```

```
    check(-2);
```

```
    cout<<"k==3\n";
```

```
    check(3);
```

```
}
```

```
catch(int g)
```

```
{
```

```
    cout<<"Caught a character exception \n";
```

```
}
```

```
catch(int j)
```

```
{
```

```
    cout<<"Caught a character exception \n";
```

```
}
```

```
catch(double s)

{

cout<<<"Caught a double exception \n";

}

cout<<<"\n End of main()";

}
```

Exceptions in constructors and destructors

Copy constructor is called in exception handling when an exception is thrown from the try block using throw statement. Copy constructor mechanism is applied to initialize the temporary object. In addition, destructors are also executed to destroy the object. If the exception is thrown from destructor, destructor are called only completely constructed objects.

Exception handling with constructor and destructor.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class number
```

```
(
```

```
private:
```

```
float x;
```

```
public:
```

```
number (float);
```

```
number()
```

```
{
```

```
}

~number()

{
    cout<<"\nIn destructor";
}

void operator ++(int) //postfix notation
{
    x++;
}

void operator --(int) //prefix notation
{
    --x;
}

void show()
{
    cout<<"\n x="<<x;
}

};

number::number(float k)
{
    if(k==0)
        throw number();
    else
```

```
x=k;
```

```
}
```

```
void main()
```

```
{
```

```
try
```

```
{
```

```
number n(2.4);
```

```
cout<<"\n before incrementation:";
```

```
n.show();
```

```
cout<<"\n after incrementation:";
```

```
n++; // postfix increment
```

```
n.show();
```

```
cout<<"\n after decrementation:";
```

```
--n; //prefix decrement
```

```
n.show();
```

```
number n1(0);
```

```
}
```

```
catch(number)
```

```
{
```

```
cout<<"\n invalid number";
```

```
exit(1);
```

```
}
```

```
}
```

```
}
```

Output

Before incrementation:

X=2.4

After incrementation:

X=3.4

After decrementation:

X=2.4

In destructor

In destructor

Invalid number

Explanation:

In the above program, the operators ++ and - - are overloaded. The class number() has two constructors and one destructor. The one argument constructor is used to initialize data member x with the value given by the user. The overloaded operators are used to increase and decrease the value of object. When the user specifies the zero value, an exception is thrown from the constructor. The exception is caught by the catch() statement. In function main() two objects N and N1 are declared. The exception is thrown when object N1 is created. When exception is thrown control goes to catch() block. The catch() block terminates the program. The destructor is also invoked.

CONTROLLING UNCAUGHT EXCEPTIONS

C++ has the following functions to control uncaught exceptions.

(1) terminate() function

In case exception handler is not defined when exception is thrown terminate() function is invoked. Implicitly the abort() function is invoked.

write a program to catch uncaught exception

```
#include<iostream.h>

class one{};

class two{};

void main()

{

try

{

cout<<"an uncaught exception \n";

throw two();

}

catch(one)

{

cout<<"exception for one";

}

}
```

Explanation:

In the above program, an exception is thrown for class two using the statement `throw two()`. The `catch()` block is defined to handle exception for class one and it contains an argument of class one type. When an exception is thrown it does not find a match hence the program is terminated. For termination, `abort()` function is called implicitly by the compiler.

(2) `set_terminate()` function

the `set_terminate()` function is used to transfer the control to specified error handling function. The `set_terminate()` function requires only one argument that is nothing but a function name where control is transferred. It returns nothing. When no function is specified,

the program is terminated. For termination, abort() function is called implicitly by the compiler.

write a program to demonstrate the use of set_terminate() function.

```
#include<iostream.h>

#include<except.h>

class one{};

class two{};

void skip()

{

cout<<"\n function skip() is invoked";

}

void main()

{

set_terminate(skip);

try

{

cout<<"\n throwing exception:";

throw(two);

}

catch(one)
```

```
{  
  
    cout<<"\n an exception of one is caught";  
  
}
```

Exception and operator overloading

Exception handling can be used with operator-overloaded functions. The following program illustrates the same.

Write a program to throw exception from overloaded operator function.

```
#include<iostream.h>  
  
#include<process.h>  
  
class number  
{  
  
    int x;  
  
    public:  
  
    number()  
  
    {};  
  
    void operator - -();  
  
    void show()  
  
    {  
  
        cout<<"\n x="<<x;
```

```
}

number(int k)

{
    x=k;
}

};

void number::operator --()

{
    if(x==0) throw number();

    else --x;
}

void main()

{
    try
    {
        number n(4);

        cout<<"\n before decrementation:";

        n.show();

        while(1)

        {

            cout<<"\nafter decrementation:";
```

```
--n;  
  
n.show();  
  
}  
  
}  
  
catch(number)  
  
{  
  
cout<<"\n reached to zero";  
  
exit(1);  
  
}  
  
}
```

Output

before decrementation:

x=4

after decrementation:

x=3

after decrementation:

x=2

after decrementation:

x=1

after decrementation:

x=0

after decrementation:

Reached to zero

Explanation:

In this program, the operator - - is overloaded. This operator when used with class objects decreases the values of class members. Using while() loop the value of object N is continuously decreased. The object N is initialized with four. The operator - -() function checks the value of x(member of object N) and if value of x reaches to zero, an exception is thrown which is caught by the catch() statement.

EXCEPTION AND INHERITANCE

Write a program to throw an exception in derived class.

```
#include<iostream.h>
```

```
class abc
```

```
{
```

```
protected:
```

```
char name[15];
```

```
int age;
```

```
};
```

```
class abc: public abc
```

```
{
```

```
float height;
```

```
float weight;
```

```
public:
```

```
void getdata()
```

```
{
```

```
cout<<"\n enter name and age:";
```

```
cin>>name>>age;
```

```
if(age<=0)
```

```
throw abc();
```

```
cout<<"\n enter height and weight:";
```

```
cin>>height>>weight;
```

```
}
```

```
void show()
```

```
{
```

```
cout<<"\n name:"<<name<<"\n age : "<<age<<"years";
```

```
cout<<"\n height:"<<height<<"feets"<<"\n weight:"<<weight<<"kg.";
```

```
}
```

```
};
```

```
void main()
```

```
{
```

```
try
```

```
{
```

```
abc x;  
  
x.getdata();  
  
x.show();  
  
}  
  
catch(abc)  
  
{  
  
cout<<"\n wrong age";  
  
}
```

Output

Enter name and age: Amit 0

Wrong age

Explanation:

In the above program, the two classes ABC and abc are declared. The class ABC has two protected data members name and age. The class abc has two float data members height and weight with two member functions getdata() and show(). The class abc is derived from class ABC. The statement class abc: public ABC defines the derived class abc. In function main(), x is an object of derived class abc. The object x invokes the member function getdata() and show(). This function reads and displays data respectively. In the function getdata(), the if statement checks the age entered. If the age entered is zero or less than zero an exception is thrown. The catch() block is executed and the message “wrong age” is displayed.

Class Template and Exception handling.

The following program illustrates how exception handling with both class templates.

Write a program to show exception handling with class template.

```
#include<iostream.h>

#include<math.h>

class sq{};

template<class T>

class square

{

    T s;

public:

    square(T x)

    {

        sizeof(x) ==1 ? throw sq() : s=x*x;

    }

    void show()

    {

        cout<<"\n square : "<<s;

    }

};

void main();
```

```
{  
  
try  
  
{  
  
square <int> i(2);  
  
i.show();  
  
square <char> c('C');  
  
c.show();  
  
}  
  
catch(sq)  
  
{  
  
cout<<"\n square of character cannot be calculated";  
  
}  
  
}
```

Explanation:

In the above program, the class square uses template type of argument. Using conditional operator, size of passed argument is checked. If it is equal to one an exception is thrown, otherwise square is calculated and stored in the class data member s. the show() function displays the square on the screen. When a character type data is entered, an exception is thrown because size of character is one and square cannot be calculated.